3. PyTorch를 통한 모델 서빙

```
!pip install datasets torchserve torch-model-archiver torch-workflow-archiver nvgpu
```

```
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: torchserve in /usr/local/lib/python3.12/dist-packages (0.12.0)
Requirement already satisfied: torch-model-archiver in /usr/local/lib/python3.12/dist-packages (0.
Requirement already satisfied: torch-workflow-archiver in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: nvgpu in /usr/local/lib/python3.12/dist-packages (0.10.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from datase
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from da
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from d
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from datas
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: huggingface-hub>=0.24.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datase
Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dist-packages (from torchserve)
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from torchserve)
Requirement already satisfied: wheel in /usr/local/lib/python3.12/dist-packages (from torchserve)
Requirement already satisfied: enum-compat in /usr/local/lib/python3.12/dist-packages (from torch-
Requirement already satisfied: ansi2html in /usr/local/lib/python3.12/dist-packages (from nvgpu) (
Requirement already satisfied: arrow in /usr/local/lib/python3.12/dist-packages (from nvgpu) (1.4.
Requirement already satisfied: flask in /usr/local/lib/python3.12/dist-packages (from nvgpu) (3.1.
Requirement already satisfied: flask-restful in /usr/local/lib/python3.12/dist-packages (from nvgp
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages (from nvgpu) (1.17.0
Requirement already satisfied: termcolor in /usr/local/lib/python3.12/dist-packages (from nvgpu) (
Requirement already satisfied: tabulate in /usr/local/lib/python3.12/dist-packages (from nvgpu) (0
Requirement already satisfied: pynvml in /usr/local/lib/python3.12/dist-packages (from nvgpu) (13.
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from h
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from reque
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: python-dateutil>=2.7.0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: tzdata in /usr/local/lib/python3.12/dist-packages (from arrow->nvgp
Requirement already satisfied: blinker>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from fla
Requirement already satisfied: click>=8.1.3 in /usr/local/lib/python3.12/dist-packages (from flask
Requirement already satisfied: itsdangerous>=2.2.0 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: jinja2>=3.1.2 in /usr/local/lib/python3.12/dist-packages (from flas
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: werkzeug>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from fl
Requirement already satisfied: aniso8601>=0.82 in /usr/local/lib/python3.12/dist-packages (from fl
Requirement already satisfied: pytz in /usr/local/lib/python3.12/dist-packages (from flask-restful
Requirement already satisfied: nvidia-ml-py>=12.0.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from a
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aioh
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from a
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from
```

```
import torch
import torch.nn.functional as F
import matplotlib.pyplot as plt
import seaborn as sns

from torch.utils.data import DataLoader
from torch.optim import AdamW
```

```
from transformers import BatchEncoding, BertTokenizer, BertForSequenceClassification
from sklearn.metrics import confusion_matrix
from datasets import load_dataset
from tqdm import tqdm
from typing import TypedDict
```

```
dataset = load_dataset("ag_news")
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=4)
optimizer = AdamW(model.parameters(), lr=5e-5)
criterion = torch.nn.CrossEntropyLoss()

class DatasetItem(TypedDict):
    text: str
    label: str


def preprocess_data(dataset_item: DatasetItem) -> dict[str, torch.Tensor]:
    return tokenizer(dataset_item["text"], truncation=True, padding="max_length", return_tensors="


train_dataset = dataset["train"].select(range(1200)).map(preprocess_data, batched=True)
test_dataset = dataset["test"].select(range(800)).map(preprocess_data, batched=True)

train_dataset.set_format("torch", columns=["input_ids", "attention_mask", "label"])
test_dataset.set_format("torch", columns=["input_ids", "attention_mask", "label"])

train_loader = DataLoader(train_dataset, batch_size=8, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=8, shuffle=False)
```

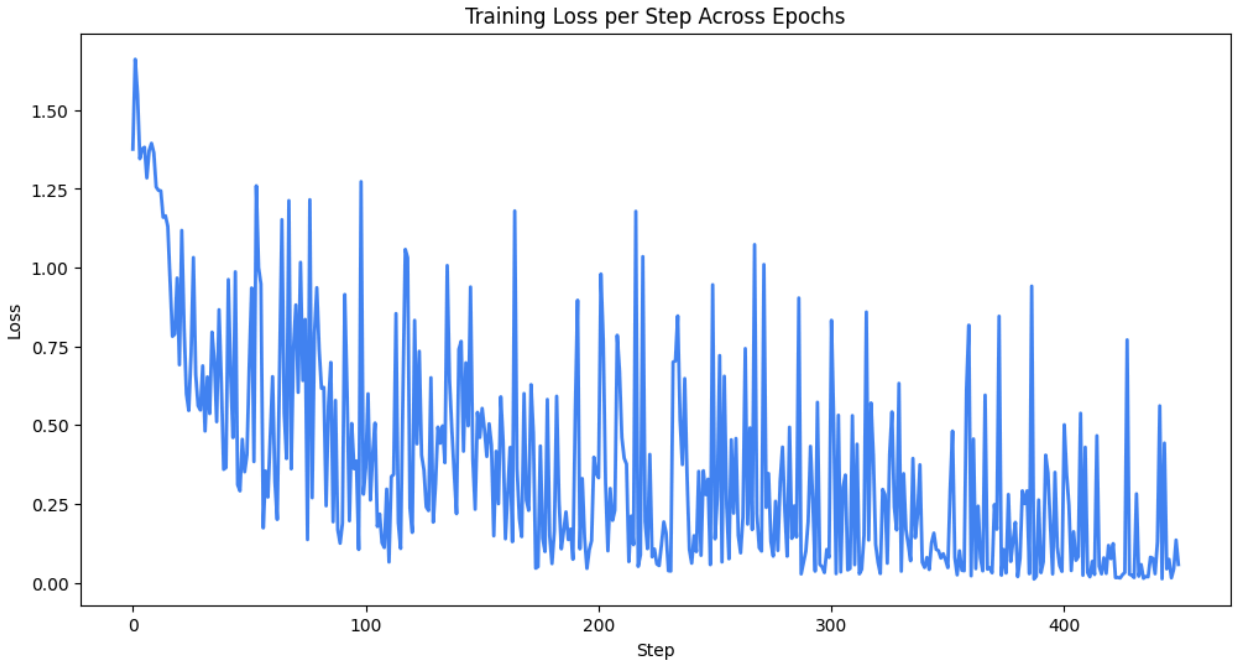
```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or dataset
  warnings.warn(
README.md:       8.07k/? [00:00<00:00, 639kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable hig
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub
data/train-00000-of-00001.parquet: 100%  18.6M/18.6M [00:01<00:00, 39.4MB/s]
data/test-00000-of-00001.parquet: 100%  1.23M/1.23M [00:00<00:00, 6.16MB/s]
Generating train split: 100%  120000/120000 [00:00<00:00, 323983.80 examples/s]
Generating test split: 100%  7600/7600 [00:00<00:00, 144587.22 examples/s]
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 1.52kB/s]
vocab.txt:       232k/? [00:00<00:00, 2.71MB/s]
tokenizer.json:       466k/? [00:00<00:00, 6.92MB/s]
config.json: 100%  570/570 [00:00<00:00, 1.45kB/s]
model.safetensors: 100%  440M/440M [00:03<00:00, 232MB/s]
Loading weights: 100%  199/199 [00:00<00:00, 399.12it/s, Materializing param=bert.pooler.dense.weight]
BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                     | Status     |
----------------------------------------+------------+-
cls.predictions.bias                    | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.seq_relationship.bias               | UNEXPECTED |
cls.seq_relationship.weight             | UNEXPECTED |
cls.predictions.transform.dense.weight  | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.transform.dense.bias    | UNEXPECTED |
classifier.weight                       | MISSING    |
classifier.bias                         | MISSING    |

Notes:
- UNEXPECTED  :can be ignored when loading from different task/architecture; not ok if you expect id
- MISSING  :those params were newly initialized because missing from the checkpoint. Consider traini
Map: 100%  1200/1200 [00:01<00:00, 660.60 examples/s]
Map: 100%  800/800 [00:01<00:00, 755.81 examples/s]
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

num_epochs = 3
losses: list[float] = []

for epoch in range(num_epochs):
    model.train()
    total_loss = 0
    for batch in tqdm(train_loader, desc=f"Epoch {epoch + 1}"):
        inputs = {key: batch[key].to(device) for key in batch}
        labels = inputs.pop("label")
        outputs = model(**inputs, labels=labels)
        loss = outputs.loss
        total_loss += loss.item()
        losses.append(loss.item())

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    average_loss = total_loss / len(train_loader)
    print(f"Epoch {epoch + 1}, Average Loss: {average_loss}")
```

```
Epoch 1: 100%|██████████| 150/150 [01:44<00:00,  1.44it/s]
Epoch 1, Average Loss: 0.6377211974561214
```

```
Epoch 2: 100%|██████████| 150/150 [01:50<00:00,  1.35it/s]
Epoch 2, Average Loss: 0.31323663098116716
Epoch 3: 100%|██████████| 150/150 [01:52<00:00,  1.34it/s]Epoch 3, Average Loss: 0.18785510438804826
```

```
plt.figure(figsize=(12, 6))
plt.plot(losses, color="#4285f4", linewidth=2)
plt.xlabel("Step")
plt.ylabel("Loss")
plt.title("Training Loss per Step Across Epochs")
plt.show()
```



모델 평가

```
model.eval()
correct = 0
total = 0

with torch.no_grad():
    for batch in tqdm(test_loader, desc="Evaluating"):
        inputs = {key: batch[key].to(device) for key in batch}
        labels = inputs.pop("label")
        outputs = model(**inputs, labels=labels)
        logits = outputs.logits
        predicted_labels = torch.argmax(logits, dim=1)
        correct += (predicted_labels == labels).sum().item()
        total += labels.size(0)

accuracy = correct / total

print("")
print(f"Test Accuracy: {accuracy * 100:.2f}%")
```

```
Evaluating: 100%|██████████| 100/100 [00:23<00:00,  4.28it/s]
Test Accuracy: 87.38%
```

```
test_input = "[Official] 'Legendary Coach Resigns → Appoints New Commander' Suwon Completes Coachir
test_input_processed = tokenizer(test_input, truncation=True, padding="max_length", return_tensors=
logits = model(**test_input_processed).logits
print(logits)
predicted_labels = torch.argmax(logits, dim=1)
```

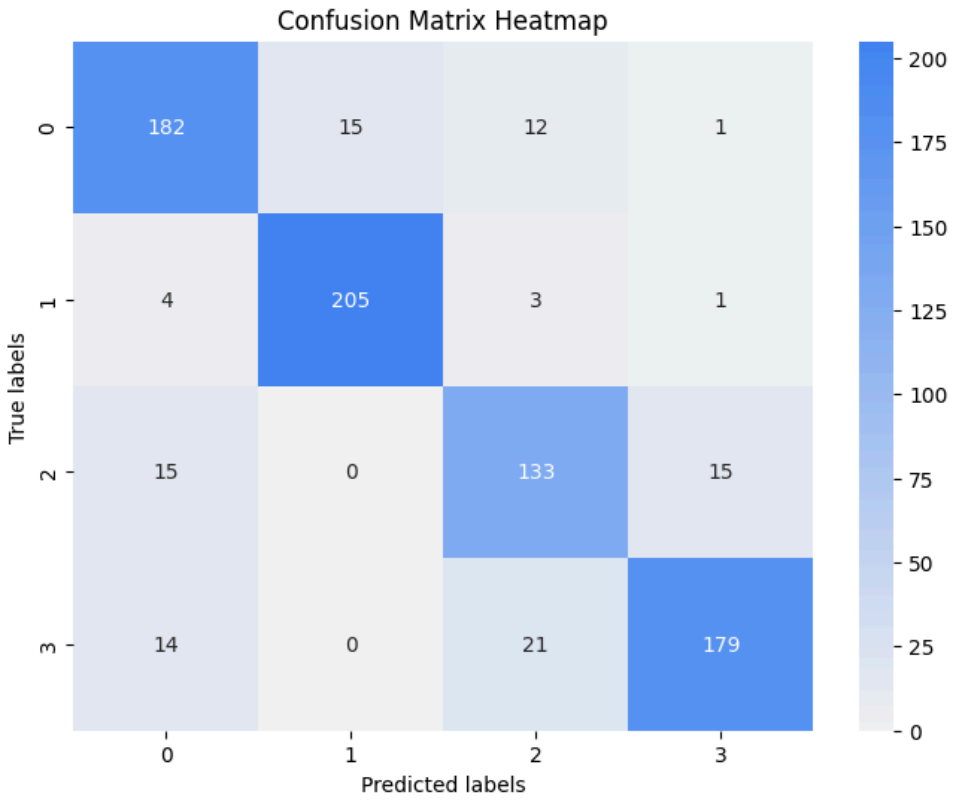
```
labeling_mapper = ["world", "sports", "business", "sci/tech"]
print(labeling_mapper[predicted_labels[0]])
```

```
tensor([[-0.1306,  3.4051, -1.5523, -2.3995]], device='cuda:0',
       grad_fn=<AddmmBackward0>)
sports
```

```
all_predictions: list[int] = []
all_labels: list[int] = []

with torch.no_grad():
    for batch in tqdm(test_loader, desc="Evaluating"):
        inputs = {key: batch[key].to(device) for key in batch}
        labels = inputs.pop("label")
        outputs = model(**inputs)
        logits = outputs.logits
        predicted_labels = torch.argmax(logits, dim=1)

        all_predictions.extend(predicted_labels.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())
```

```
Evaluating: 100%|██████████| 100/100 [00:23<00:00,  4.33it/s]
```

```
conf_matrix = confusion_matrix(all_labels, all_predictions)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt="g", cmap=sns.light_palette("#4285f4", as_cmap=True))
plt.xlabel("Predicted labels")
plt.ylabel("True labels")
plt.title("Confusion Matrix Heatmap")
plt.show()
```



모델 서빙


```
# 모델 체크포인트 저장.
model_save_path = "bert_news_classification_model.pth"
torch.save(model.state_dict(), model_save_path)
```

```
%%writefile model_handler.py
import json

import torch
from ts.context import Context
from ts.torch_handler.base_handler import BaseHandler
from transformers import BatchEncoding, BertTokenizer, BertForSequenceClassification

class ModelHandler(BaseHandler):
    def __init__(self):
        self.initialized = False
        self.tokenizer = None
        self.model = None

    def initialize(self, context: Context):
        self.initialized = True
        self.tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
        self.model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=
        self.model.load_state_dict(torch.load("bert_news_classification_model.pth"))
        self.model.to(torch.device("cuda" if torch.cuda.is_available() else "cpu"))
        self.model.eval()

    def preprocess(self, data: list[dict[str, bytearray]]) -> BatchEncoding:
        model_input_texts: list[str] = sum([json.loads(item.get("body").decode("utf-8"))["data"] fo
        inputs = self.tokenizer(model_input_texts, truncation=True, padding=True, max_length=512, r
        device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        return inputs.to(device)

    def inference(self, input_batch: BatchEncoding) -> torch.Tensor:
        with torch.no_grad():
            outputs = self.model(**input_batch)
            return outputs.logits

    def postprocess(self, inference_output: torch.Tensor) -> list[dict[str, float]]:
        probabilities = torch.nn.functional.softmax(inference_output, dim=1)
        return [{"label": int(torch.argmax(prob)), "probability": float(prob.max())} for prob in pr
```

```
Writing model_handler.py
```

```
%%writefile config.properties
inference_address=http://0.0.0.0:5000
management_address=http://0.0.0.0:5001
metrics_address=http://0.0.0.0:5002
```

```
Writing config.properties
```

```
# bert vocab 파일 (아티팩트)

!wget https://raw.githubusercontent.com/microsoft/SDNet/master/bert_vocab_files/bert-base-uncased-v
    -O bert-base-uncased-vocab.txt
```

```
--2026-04-15 03:52:46--  https://raw.githubusercontent.com/microsoft/SDNet/master/bert_vocab_files/b
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133,
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connecte
HTTP request sent, awaiting response... 200 OK
Length: 231508 (226K) [text/plain]
Saving to: 'bert-base-uncased-vocab.txt'

bert-base-uncased-v 100%[===================>] 226.08K  --.-KB/s    in 0.02s

2026-04-15 03:52:46 (13.4 MB/s) - 'bert-base-uncased-vocab.txt' saved [231508/231508]
```

```
# Torch serve archiver의 최종 파일 저장 위치.
# .mar 확장자 파일을 생성하여 패키징 하게 된다.

!mkdir -p model-store
```

```
!torch-model-archiver \
    --model-name bert_news_classification \
    --version 1.0 \
    --serialized-file bert_news_classification_model.pth \
```

```
    --handler ./model_handler.py \
    --extra-files "bert-base-uncased-vocab.txt" \
    --export-path model-store \
    -f
```

```
%%script bash --bg
# 서버를 백그라운드에서 실행

PYTHONPATH=/usr/lib/python3.10 torchserve \
    --start \
    --ncs \
    --ts-config config.properties \
    --model-store model-store \
    --models bert_news_classification=bert_news_classification.mar \
    --disable-token-auth
```

```
# 아래와 같이 뜨면 정상 실행.
# {
#   "status": "Healthy"
# }

!curl -X GET localhost:5000/ping
```

```
curl: (7) Failed to connect to localhost port 5000 after 0 ms: Connection refused
```

```
%%shell

# 모델 실제 평가를 위해 외부 뉴스 기사 데이터 셋 파일 생성.

cat > request_sports.json <<EOF
{
  "data": [
    "Bleary-eyed from 16 hours on a Greyhound bus, he strolled into the stadium running on fumes. H
  ]
}
EOF

cat > request_business.json <<EOF
{
  "data": [
    "DETROIT — America maintained its love affair with pickup trucks in 2023 — but a top-selling ve
  ]
}
EOF

cat > request_sci_tech.json <<EOF
{
  "data": [
    "OpenVoice comprises two AI models working together for text-to-speech conversion and voice tor
  ]
}
EOF
```

```
# 스포츠 기사 평가 (레이블: 1)
!curl -X POST \
    -H "Accept: application/json" \
    -T "request_sports.json" \
    http://localhost:5000/predictions/bert_news_classification
```

```
curl: (7) Failed to connect to localhost port 5000 after 0 ms: Connection refused
```

```
# 비즈니스 기사 평가 (레이블: 2)
!curl -X POST \
    -H "Accept: application/json" \
    -T "request_business.json" \
    http://localhost:5000/predictions/bert_news_classification
```

```
curl: (7) Failed to connect to localhost port 5000 after 0 ms: Connection refused
```

```
# 테크 기사 평가 (레이블: 3)
!curl -X POST \
    -H "Accept: application/json" \
    -T "request_sci_tech.json" \
    http://localhost:5000/predictions/bert_news_classification
```

```
curl: (7) Failed to connect to localhost port 5000 after 0 ms: Connection refused
```

ngrok을 통한 외부 연결

ngrok은 로컬에 서비스되는 데몬/서버에 외부 도메인을 연결시켜, 외부에서 접근하기 쉽도록 만들어주는 서비스입니다.

5000번으로 호스팅한 서빙 서버 접근을 위해 아래 코드에서는 ngrok을 연결합니다.

```
!pip install pyngrok
```

```
Collecting pyngrok
  Downloading pyngrok-8.0.0-py3-none-any.whl.metadata (8.2 kB)
Requirement already satisfied: PyYAML>=5.1 in /usr/local/lib/python3.12/dist-packages (from pyngrok)
Downloading pyngrok-8.0.0-py3-none-any.whl (24 kB)
Installing collected packages: pyngrok
Successfully installed pyngrok-8.0.0
```